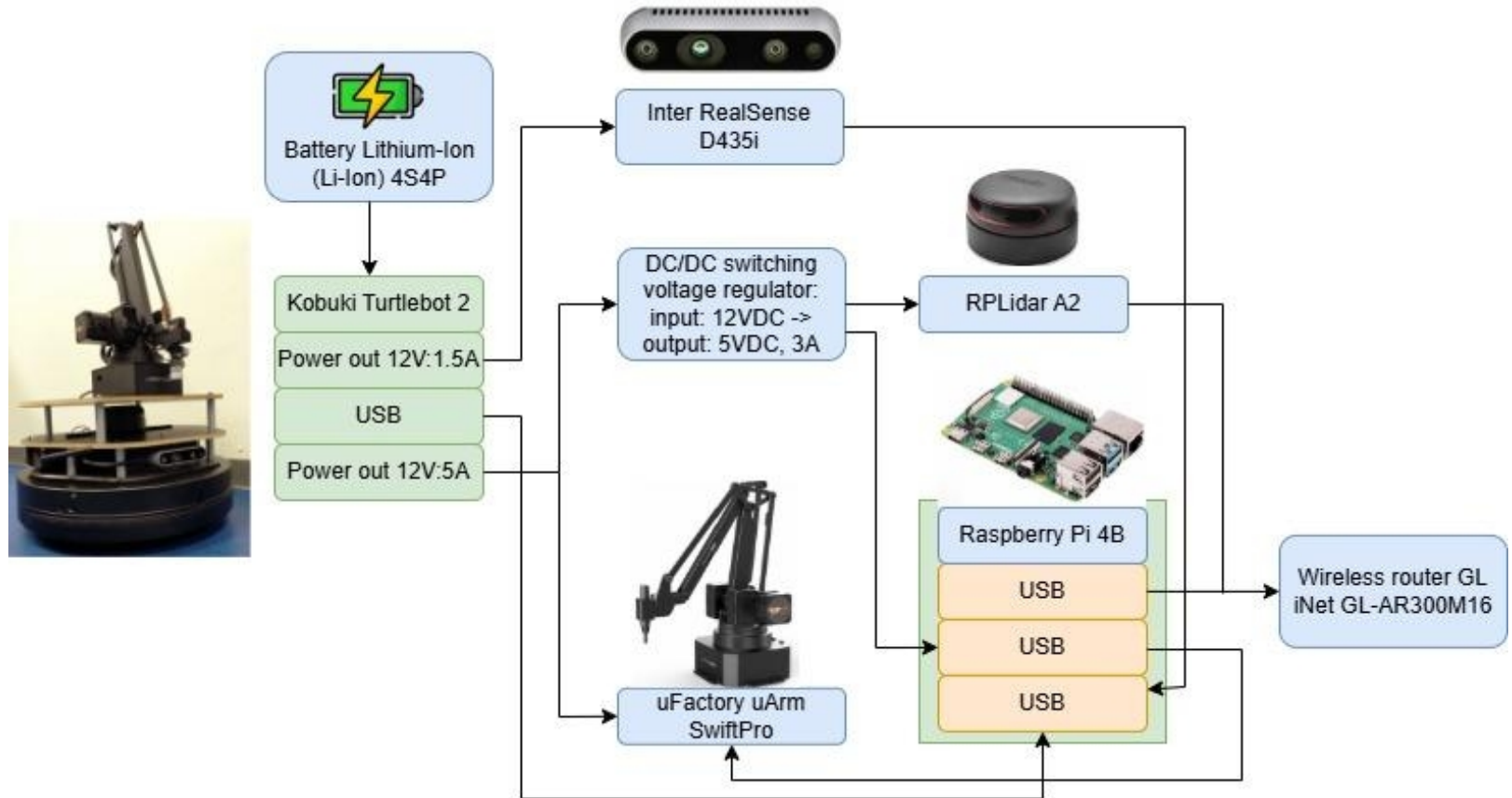


Control Architecture



Forward Kinematics

3D Cartesian coordinates of the end-effector (in mm) are:

$$x = (13.2 - 142\sin\theta_2 + 158.8\cos\theta_3 + 56.5)\cos\theta_1$$

$$y = (13.2 - 142\sin\theta_2 + 158.8\cos\theta_3 + 56.5)\sin\theta_1$$

$$z = -108 - 142\cos\theta_2 - 158.8\sin\theta_3 + 72.2$$

Here, $\theta_1 = J_1$, $\theta_2 = J_2$, $\theta_3 = J_3$, $\theta_4 = J_4$

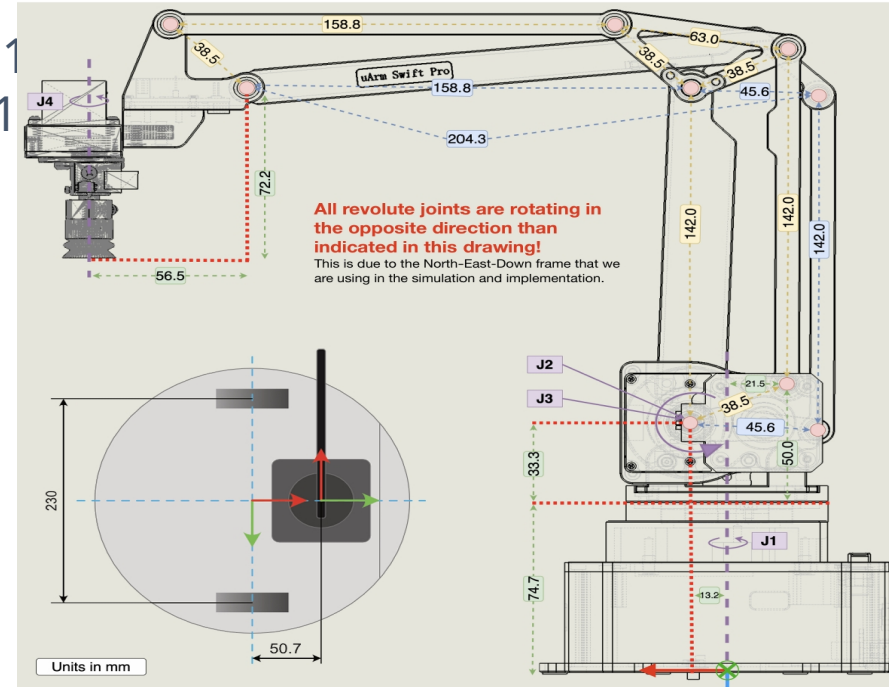
Homogeneous Transformation

World to Mobile Base 0H_1

Mobile Base to Manipulator Base 1H_2

Manipulator base to the end-effector 2H_3

World frame to the end-effector ${}^0H_3 = {}^2H_3 \cdot {}^1H_2 \cdot {}^0H_1$



Recursive Algorithm for Task-Priority Inverse Kinematics

Initialize: $dq = 0$, $P = I \rightarrow$ Start with zero motion and full DOF

For each task:

Compute task error: $err = \text{desired} - \text{current} \rightarrow$ Find how far we are from goal

Apply gain of 0.8 $\rightarrow$ Control how fast we react to the error

Get task Jacobian $\rightarrow$ Understand how motion affects the task

Project Jacobian into null space $\rightarrow$ Prevent conflicts with higher-priority tasks

Solve dq with WDLS: $\lambda = 0.01$, custom weights for smooth, stable motion, prefer specific joints

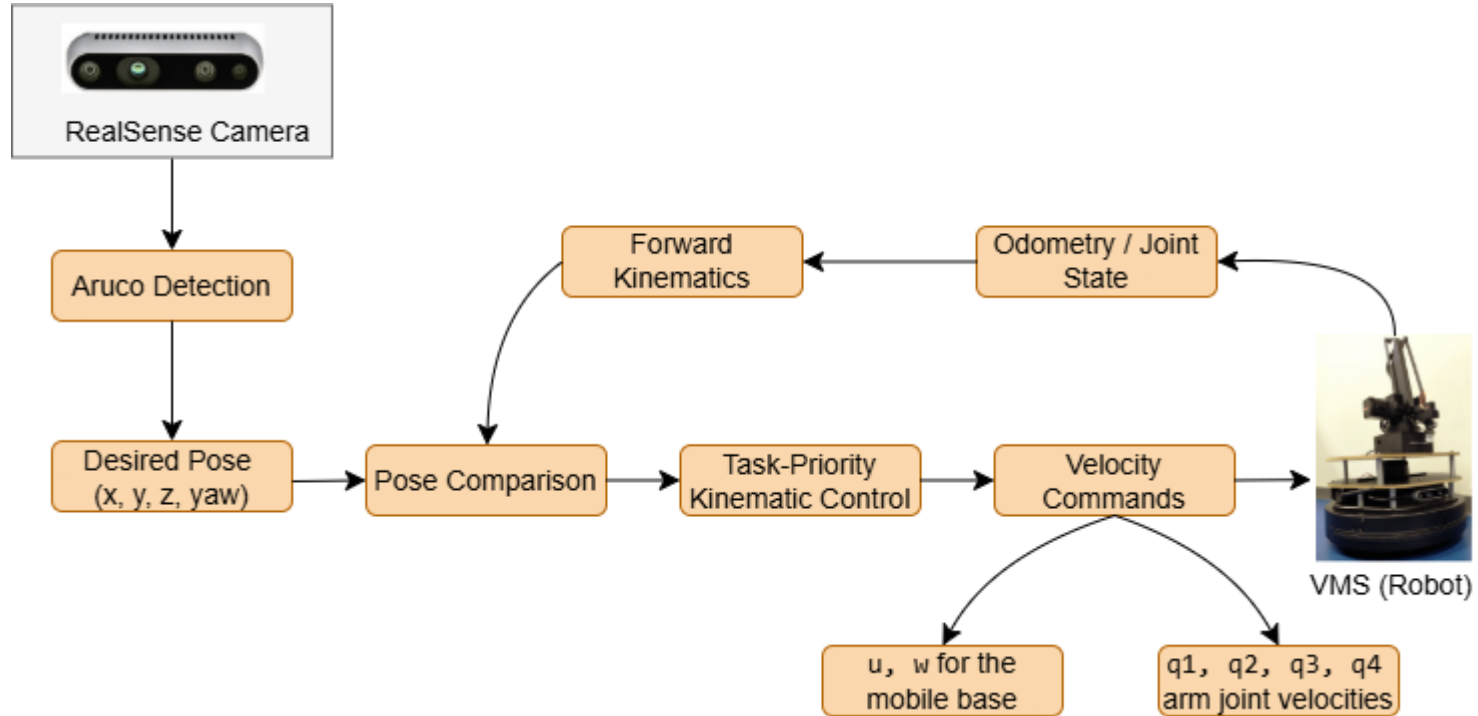
Update dq: $dq += W_DLS @ (err - J @ dq) \rightarrow$ Add this task's motion to final command

Compute pseudoinverse: $J_pinv = pinv(\bar{J}) \rightarrow$ Remove used motion from available space.

Update null space P $P = P - J_pinv @ \bar{J} \rightarrow$ Reserve remaining DOF for next tasks

Scale velocity $\rightarrow$ Ensure safe, physically feasible motion

Kinematic Architecture



Task setups

Task index of 0 -> Joint Limits

End-effector position

Task index of 1 -> Joint Limits

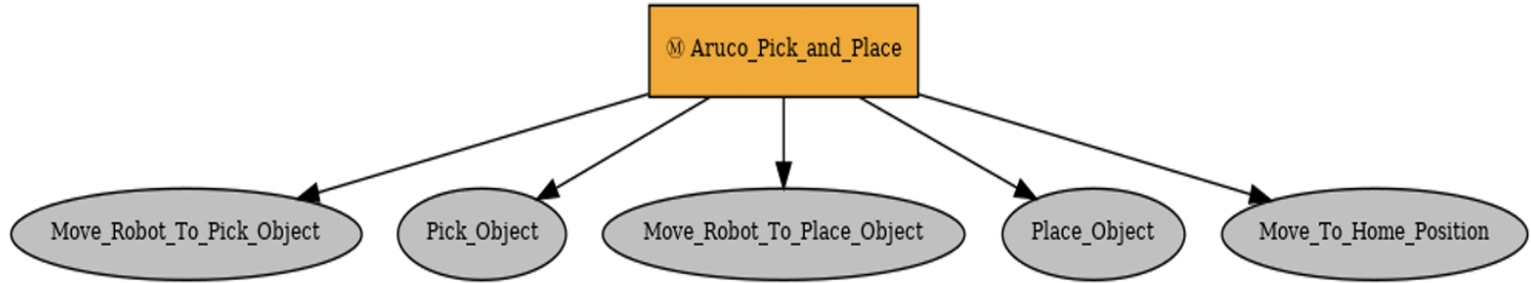
Joint angle at joint 1 (placed above robots base)

End-effector position

Task index of 2 -> Joint Limits

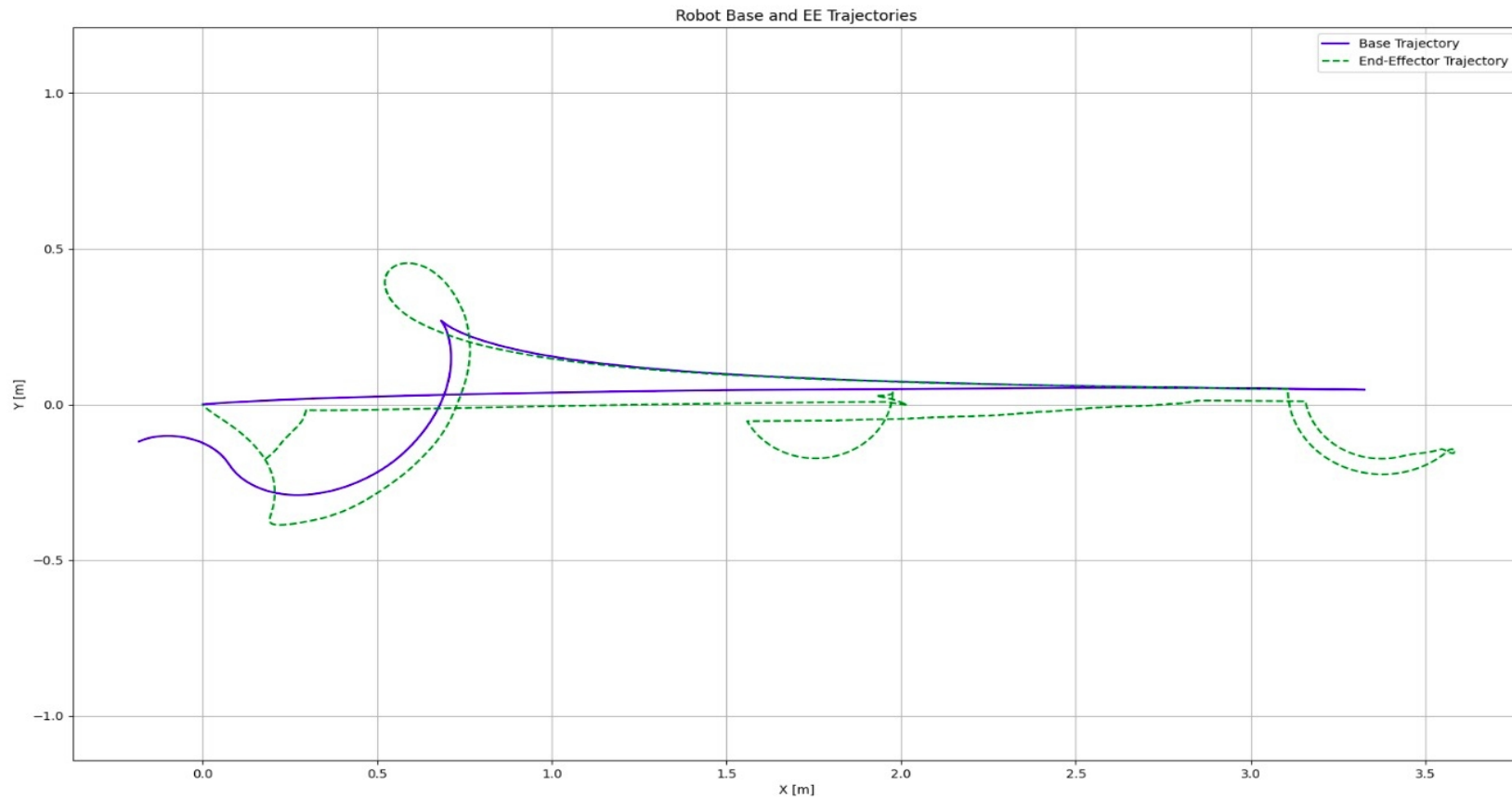
Joint angle at joint 1 (placed above robots base)

Behavior Tree

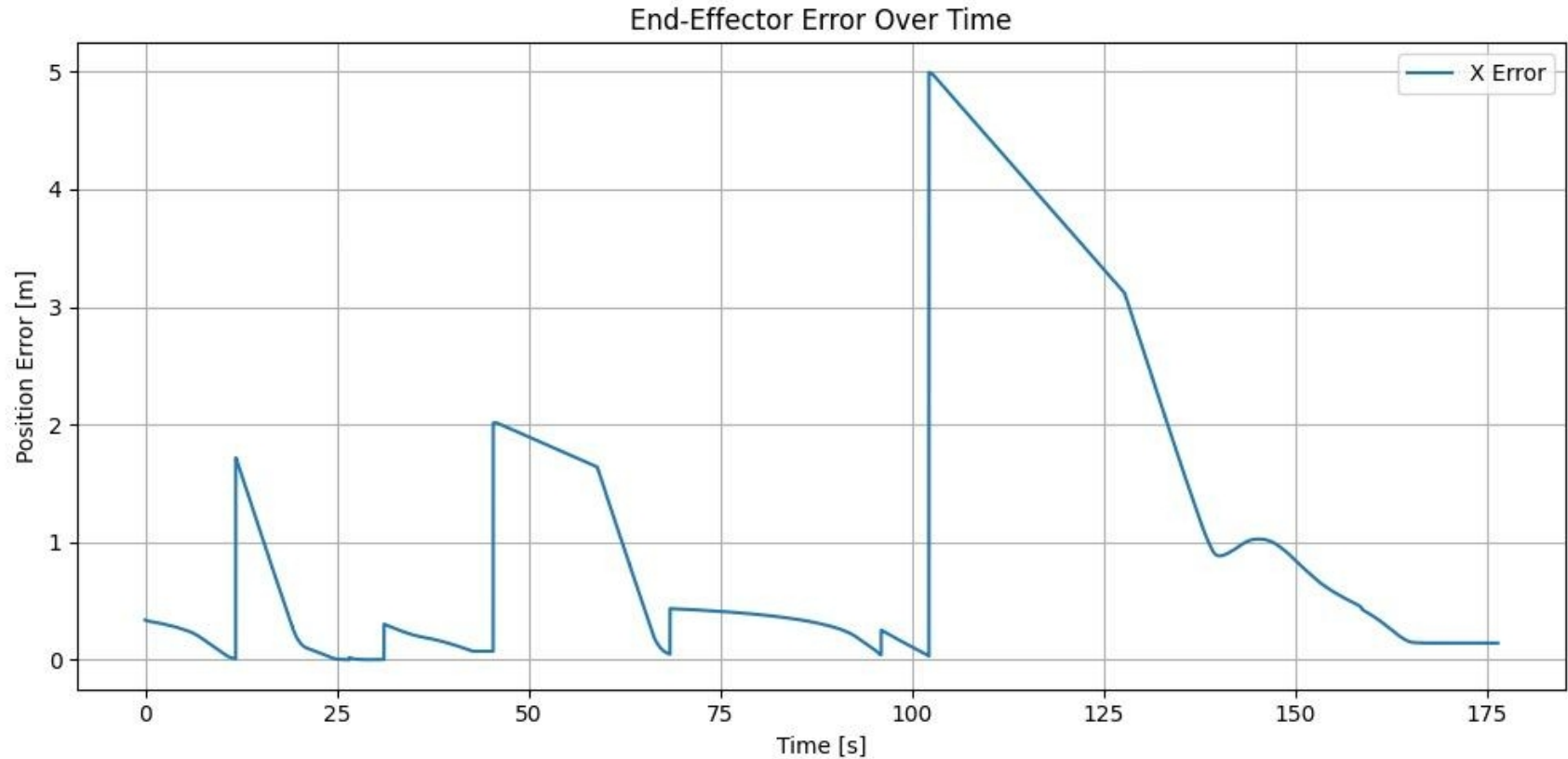


Node	Purpose	Task Index
MoveRobotToPick	Align robot to box using AruCo	0
PickPoints	Move arm down and pick using gripper	0
MoveRobotToPlace	Rotate and move robot to place location	2 -> 1 -> 0
PlacePoints	Lower object and deactivate gripper	0
MoveToHome	Return to original base and arm pose	2 -> 1

Base and EE Trajectory



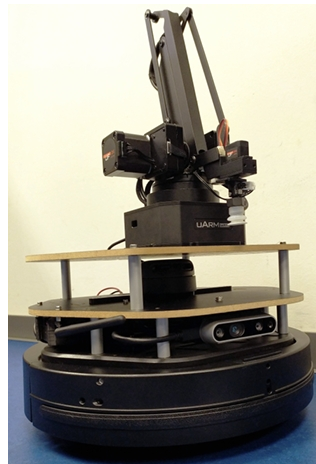
EE Error Over Time



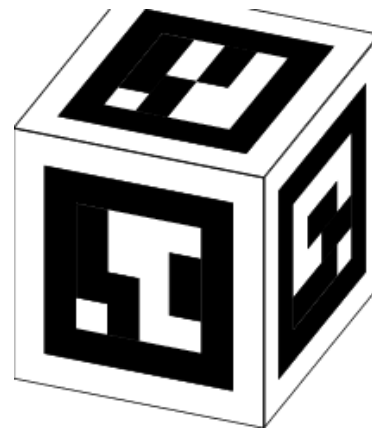
Key Components & System Overview



Camera



Turtlebot

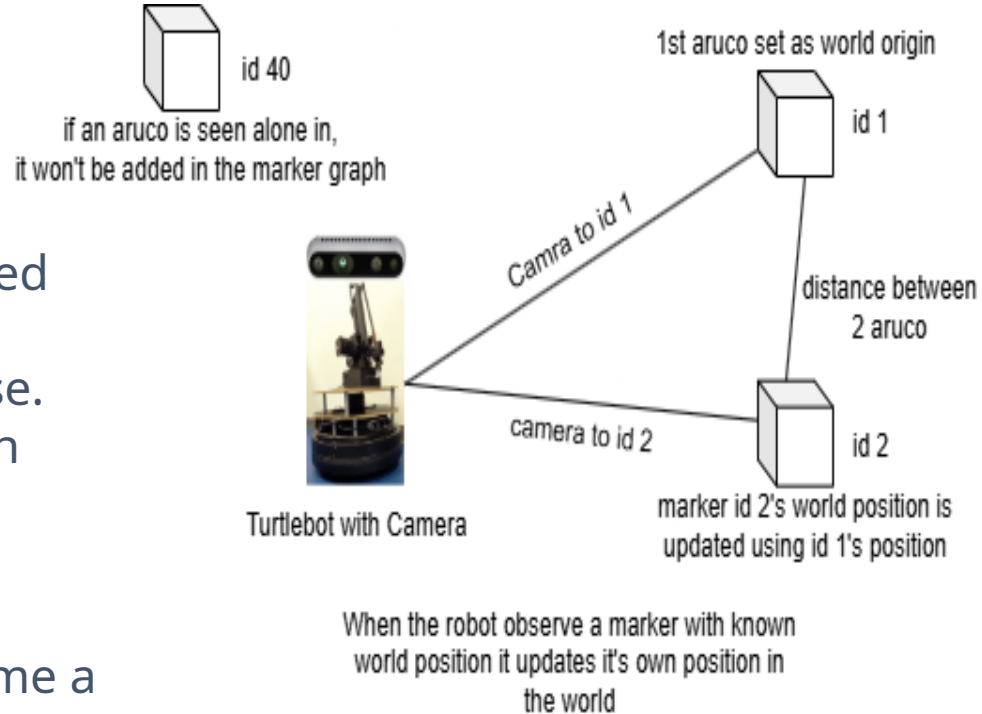


Aruco box

- TF (Transform) tree manages how frames like camera, base_link, odom, and world are connected.
- RViz, Visualization tool to confirm correct positions of the robot and markers in 3D space.
- Robot sees ArUco markers using onboard camera
- Unknown markers → saved as global pose
- Known markers → used to update robot pose

Pose Estimation Pipeline

- Marker detection via camera
- TF created from camera → marker
- If marker already known:
 - Estimate robot pose using chained transforms.
 - world → marker → camera → base.
- If new marker seen along with known marker:
 - Add to graph using relative transform.
- Robot localization is updated each time a known marker is seen.



Marker Detection & Mapping Transformations

Step	Transformation
Setting world origin	$T_{\text{world_to_marker}} = \text{Identity}$
Estimating pose of marker	$T_{\text{camera_to_marker}} \rightarrow \text{solvePnP}()$ $T_{\text{marker_to_camera}} = \text{inverse}(T_{\text{camera_to_marker}})$
Computing camera pose in world	$T_{\text{world_to_camera}} = T_{\text{world_to_marker}} \times T_{\text{marker_to_camera}}$
Transforming to robot base	$T_{\text{world_to_base}} = T_{\text{world_to_camera}} \times T_{\text{camera_to_base}}$

Transformation between two ArUcos seen at the same time:

$T_{\text{markerID_to_camera}} = \text{inverse}(T_{\text{camera_to_markerID}})$

$T_{\text{marker1_to_marker2}} = T_{\text{marker1_to_camera}} \times T_{\text{camera_to_marker2}}$

$T_{\text{world_to_marker2}} = T_{\text{world_to_marker1}} \times T_{\text{marker1_to_marker2}}$

Robot Localization via Known Marker

Known:

$T_{\text{world_to_marker}}$ (from the map)

$T_{\text{camera_to_marker}}$ (from solvePnP)

$T_{\text{marker_to_camera}} = \text{inverse}(T_{\text{camera_to_marker}})$

Compute:

$T_{\text{world_to_camera}} = T_{\text{world_to_marker}} \times T_{\text{marker_to_camera}}$

$T_{\text{world_to_base}} = T_{\text{world_to_camera}} \times T_{\text{camera_to_base}}$

Pose Estimation via Marker Fusion

1. Robot detects **multiple known markers** in the camera frame.
2. For each marker, compute the robot's pose:

$$T_{\text{world} \rightarrow \text{base}} = T_{\text{world} \rightarrow \text{marker}} \times T_{\text{marker} \rightarrow \text{camera}} \times T_{\text{camera} \rightarrow \text{base}}$$

3. Get one robot pose from **each marker**.

Fusion Strategy:

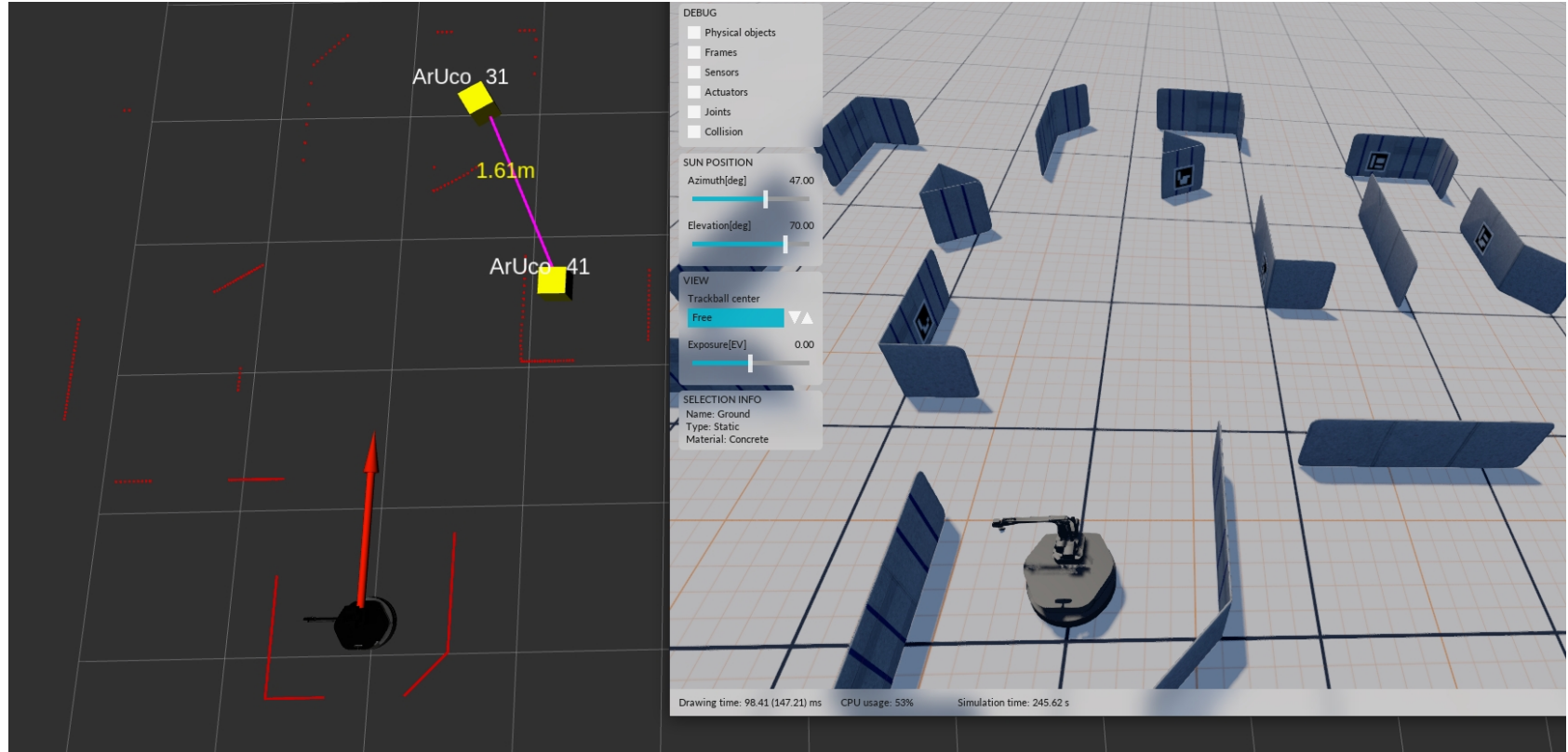
Compute the **average of all transformations**.

- **Translation:** Average all (x, y, z) positions.
- **Rotation:** Use **eigen-decomposition** to find average quaternions.

Advantages

4. Robust to detection noise and marker errors.
5. Improves as more markers become visible.

Marker Detection



Marker Initialization & Logging

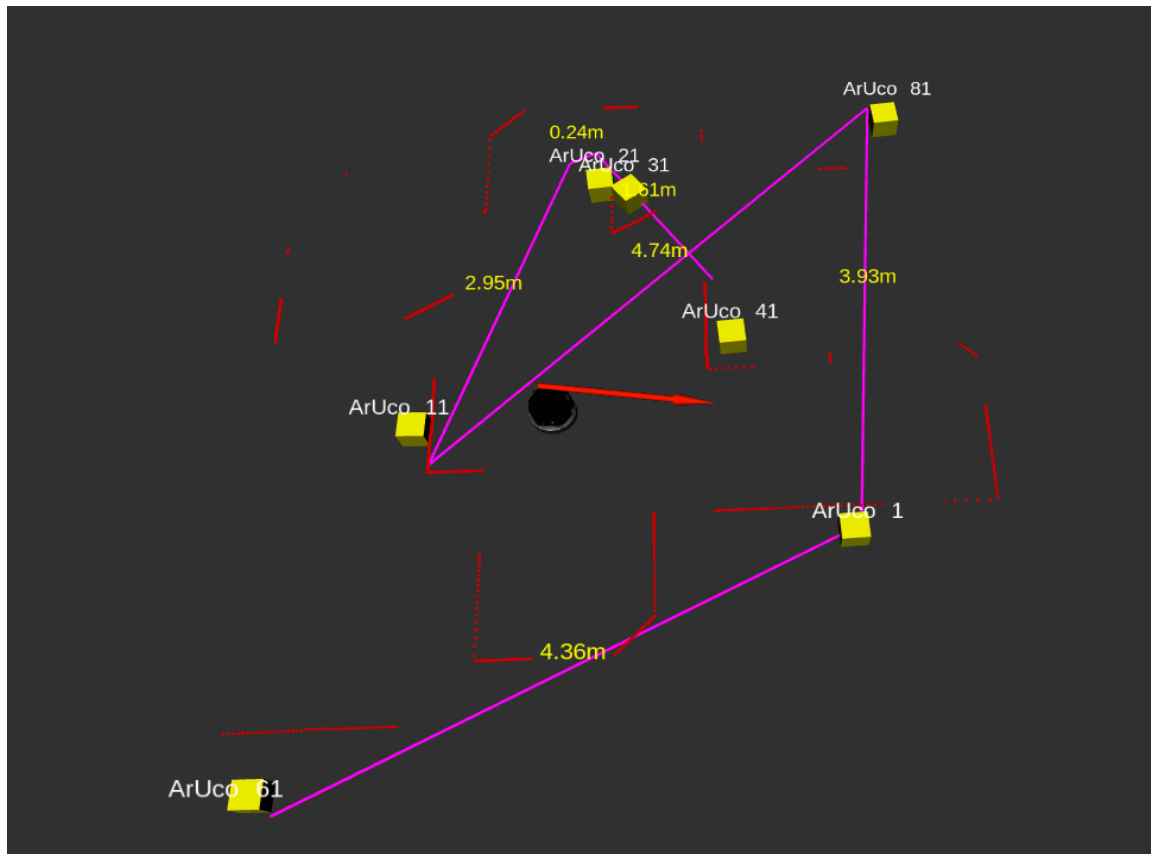
- First marker → world origin
- New markers added to graph
- Pose logged (x, y, z, yaw)
- Graph expands incrementally

```
[WARN] [1747993092.942965]: First marker 41 set as world origin.
[WARN] [1747993092.951471]: Marker 31 stored.
[INFO] [1747993092.954722]: ===== Marker Graph Structure =====
[INFO] [1747993092.956460]: Total nodes: 2
[INFO] [1747993092.958110]:   Node ID 41: (1.79, 1.62, -0.16)
[INFO] [1747993092.960130]:   Node ID 31: (2.42, 3.10, -0.16)
[INFO] [1747993092.961840]: Total edges: 0
[INFO] [1747993092.964358]: =====
[INFO] [1747993092.978147]: New marker(s) used! Total contributing markers: 1
[INFO] [1747993092.979986]:
===== ArUco-Based Robot Pose =====
Pose (world_ned):
Position: x = 3.00, y = -0.77, z = -0.20
Yaw (deg): 90.01
=====

[INFO] [1747993093.013727]: New marker(s) used! Total contributing markers: 2
[INFO] [1747993093.015272]:
===== ArUco-Based Robot Pose =====
Pose (world_ned):
Position: x = 3.00, y = -0.77, z = -0.20
Yaw (deg): 90.01
=====
```

Full map

- All markers placed and detected
- Pink lines = pairwise marker connections
- Distances (m) displayed between connected markers
- Full map used for robot localization in world frame
- Robot pose estimated using multiple visible markers



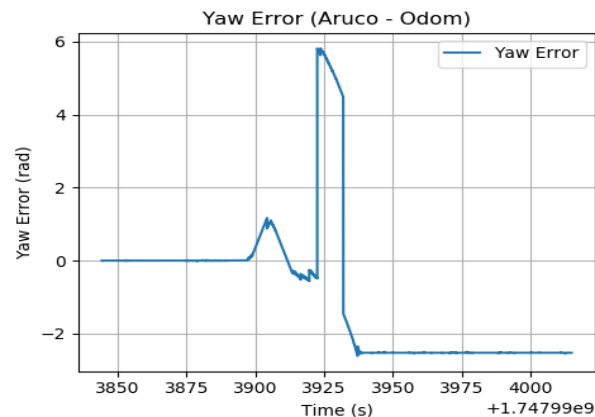
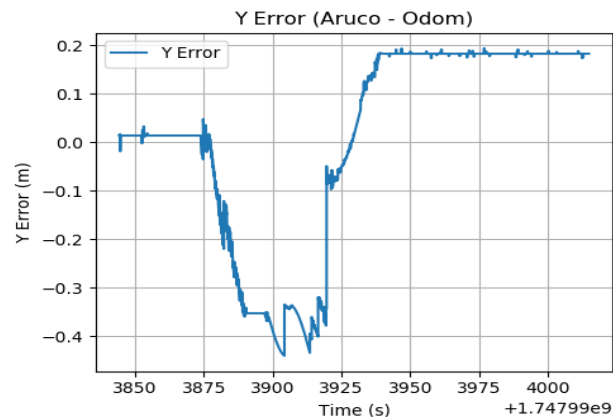
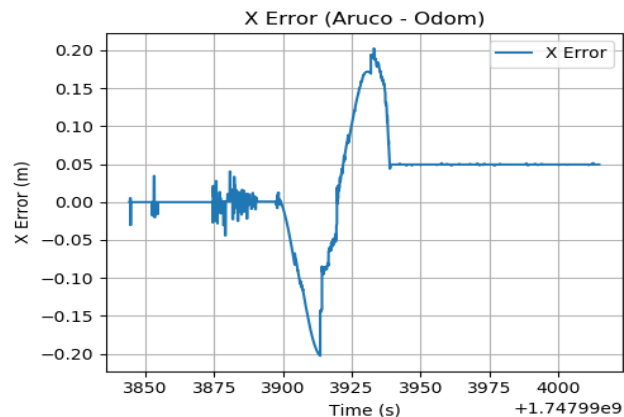
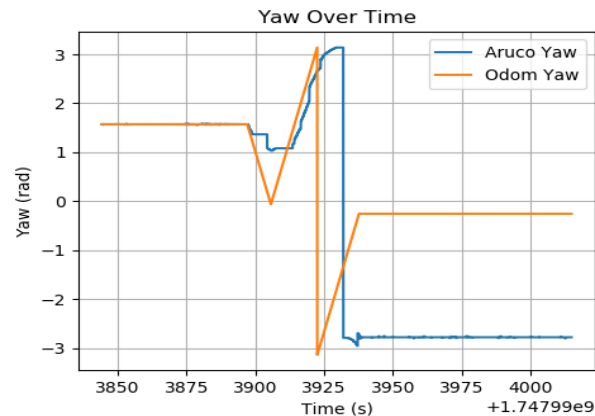
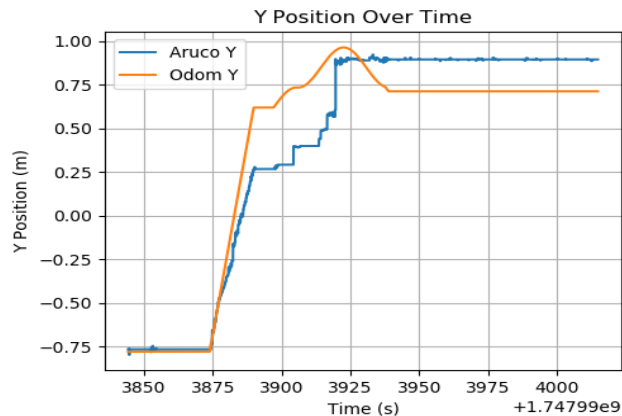
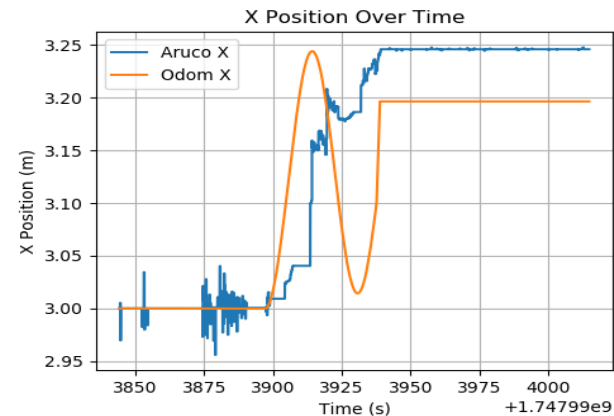
Graph Growth & Pose Update Logs

- Marker graph grows with each detection
- Edge list expands as new marker pairs are observed
- Robot pose (x, y, z, yaw) is updated live
- Logs confirm graph connectivity and localization
- Example: Pose flips after marker 61 is added (yaw jumps from $+166^\circ$ to -160°)

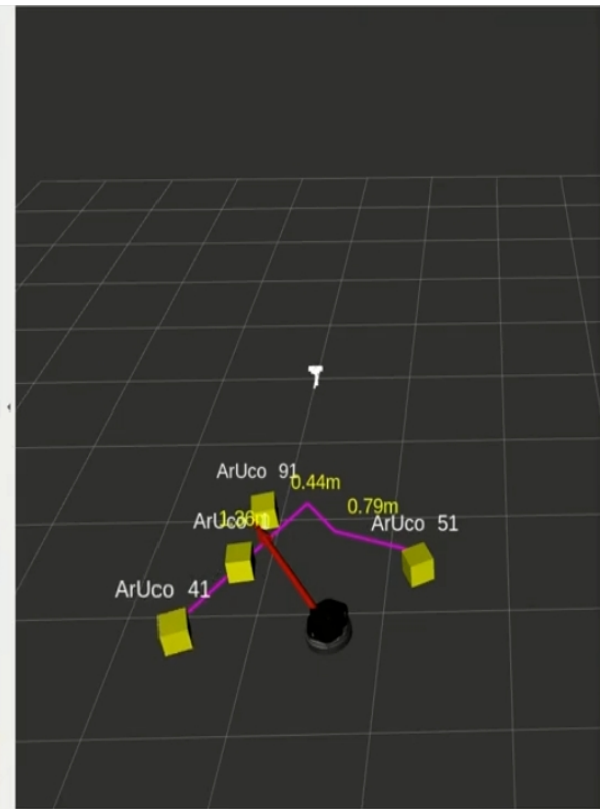
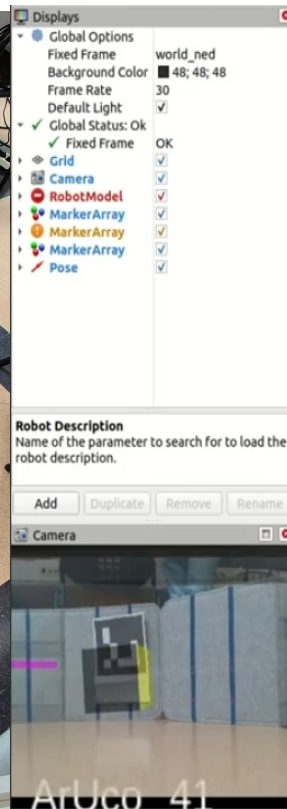
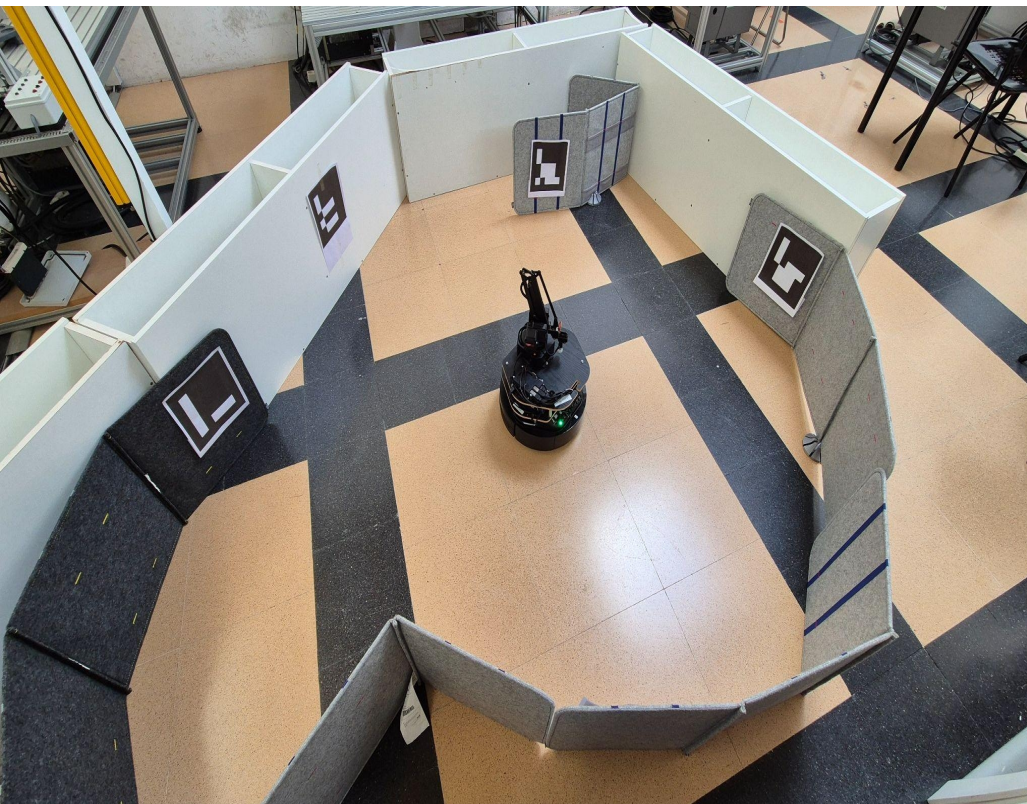
```
[INFO] [1747993923.749532]: ===== Marker Graph Structure =====
[INFO] [1747993923.751138]: Total nodes: 6
[INFO] [1747993923.754069]: Node ID 41: (1.77, 1.03, -0.16)
[INFO] [1747993923.756660]: Node ID 31: (2.24, 2.57, -0.16)
[INFO] [1747993923.758197]: Node ID 21: (2.44, 2.72, -0.16)
[INFO] [1747993923.759778]: Node ID 11: (4.15, 0.57, -0.16)
[INFO] [1747993923.761119]: Node ID 81: (0.07, 2.84, -0.16)
[INFO] [1747993923.762816]: Node ID 1: (1.25, -0.73, -0.16)
[INFO] [1747993923.764596]: Total edges: 5
[INFO] [1747993923.766129]: Edge: 41 <--> 31
[INFO] [1747993923.767476]: Edge: 31 <--> 21
[INFO] [1747993923.768272]: Edge: 31 <--> 81
[INFO] [1747993923.769258]: Edge: 21 <--> 81
[INFO] [1747993923.770306]: Edge: 41 <--> 81
[INFO] [1747993923.771196]: =====
[INFO] [1747993923.781960]: New marker(s) used! Total contributing markers: 6
[INFO] [1747993923.784918]: =====
===== ArUco-Based Robot Pose =====
Pose (world_ned):
Position: x = 3.18, y = 0.90, z = -0.19
Yaw (deg): 166.05
=====

[WARN] [1747993932.003705]: Marker 61 stored.
[INFO] [1747993932.005874]: ===== Marker Graph Structure =====
[INFO] [1747993932.008401]: Total nodes: 7
[INFO] [1747993932.010475]: Node ID 41: (1.77, 1.03, -0.16)
[INFO] [1747993932.012488]: Node ID 31: (2.24, 2.57, -0.16)
[INFO] [1747993932.015703]: Node ID 21: (2.44, 2.72, -0.16)
[INFO] [1747993932.018074]: Node ID 11: (4.15, 0.57, -0.16)
[INFO] [1747993932.019586]: Node ID 81: (0.07, 2.84, -0.16)
[INFO] [1747993932.022377]: Node ID 1: (1.32, -0.77, -0.16)
[INFO] [1747993932.025326]: Node ID 61: (5.46, -1.86, -0.16)
[INFO] [1747993932.027227]: Total edges: 6
[INFO] [1747993932.029146]: Edge: 41 <--> 31
[INFO] [1747993932.030572]: Edge: 31 <--> 21
[INFO] [1747993932.032439]: Edge: 31 <--> 81
[INFO] [1747993932.034402]: Edge: 21 <--> 81
[INFO] [1747993932.035854]: Edge: 41 <--> 81
[INFO] [1747993932.037020]: Edge: 41 <--> 1
[INFO] [1747993932.037458]: New marker(s) used! Total contributing markers: 7
[INFO] [1747993932.037979]: =====
[INFO] [1747993932.039074]: =====
===== ArUco-Based Robot Pose =====
Pose (world_ned):
Position: x = 3.21, y = 0.91, z = -0.20
Yaw (deg): -160.03
=====
```

Pose Comparison (Simulation): ArUco vs Odometry

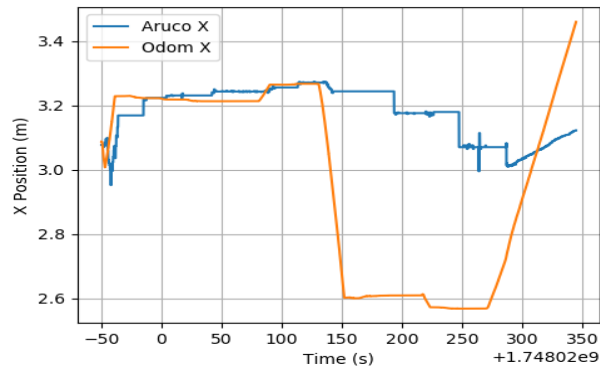


Real world Implementation

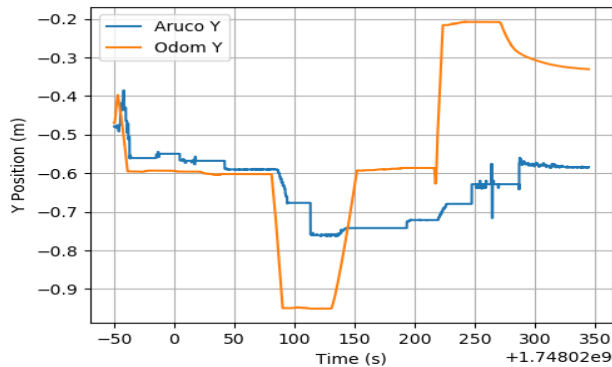


Pose Comparison (Real Robot): ArUco vs Odometry

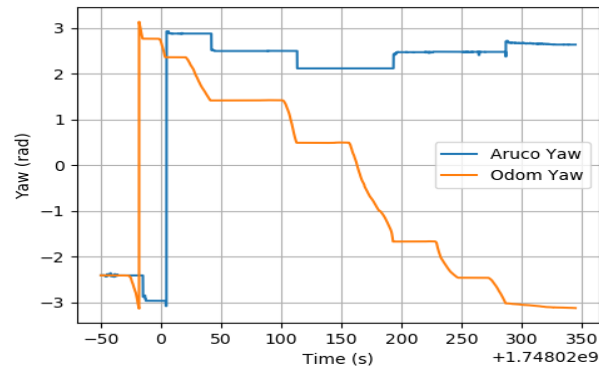
X Position Over Time



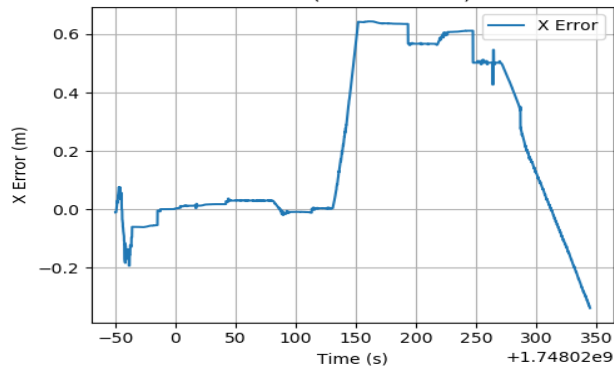
Y Position Over Time



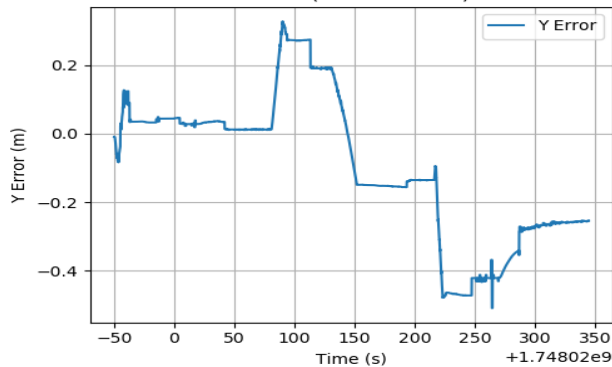
Yaw Over Time



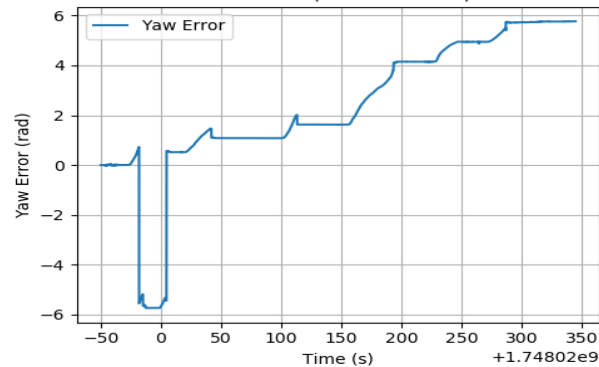
X Error (Aruco - Odom)



Y Error (Aruco - Odom)



Yaw Error (Aruco - Odom)



Problem Statement

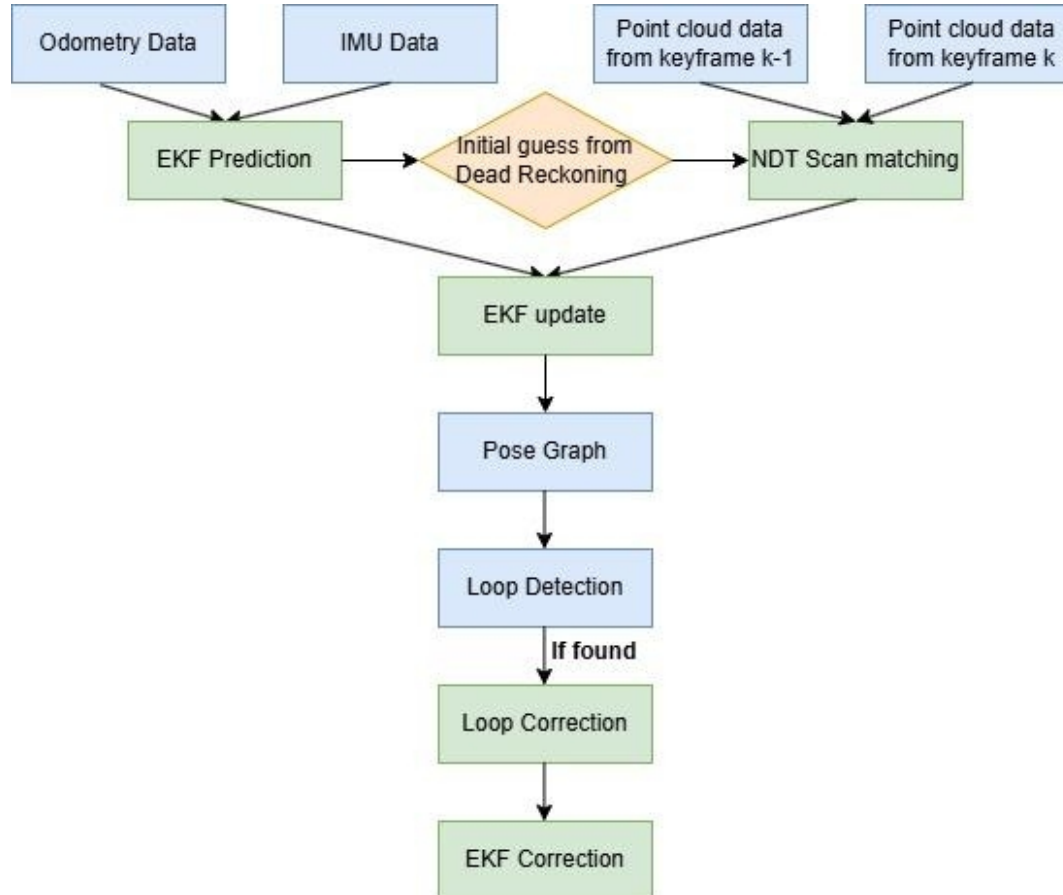
Challenge:

- How to achieve accurate and reliable localization in GPS-denied environments?
- How to overcome odometry drift caused by wheel slippage and sensor noise in long trajectories?

Solution:

- Combines dead reckoning (wheel + IMU) with NDT scan matching for pose estimation.
- Implements Pose-based EKF to fuse prediction and correction steps.
- Incorporates a keyframe-based loop closure detector.

System Overview



NDT Scan Matching and EKF Fusion Flow

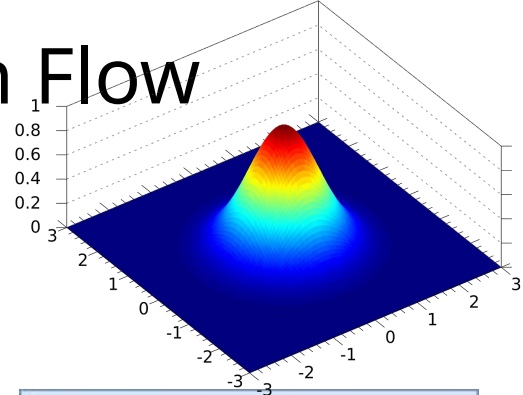
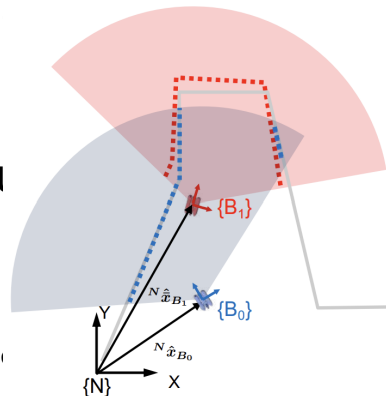
NDT stands for Normal Distributions Transform, used to align LiDAR scans.

The reference scan is divided into voxels; each voxel is modeled as a Gaussian.

The current scan is aligned by maximizing how well its points fit these distributions.

An optimizer refines the robot's pose from an initial guess (e.g., odometry).

The process stops when change is small (0.01) or max steps reached (30 iterations).



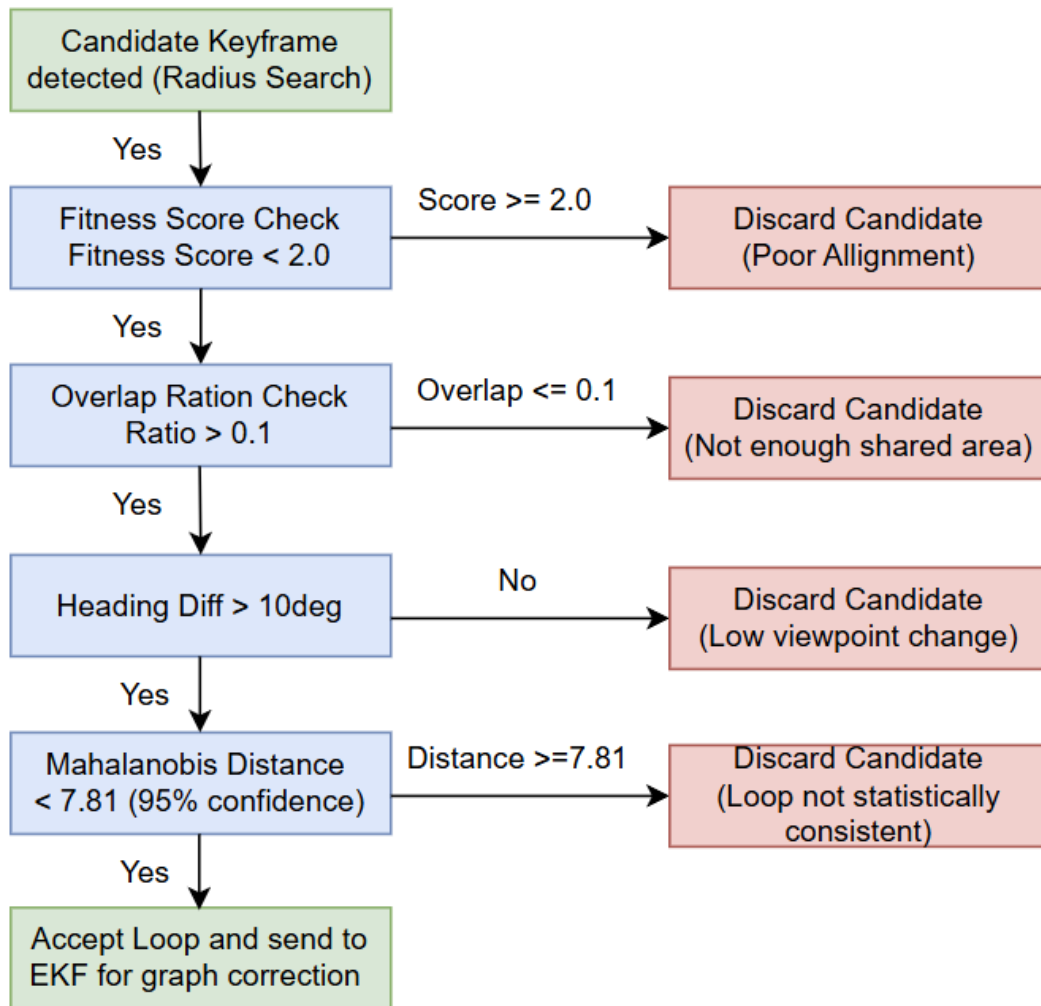
EKF Prediction Step
(Dead Reckoning: Wheel+IMU)

NDT Scan Matching

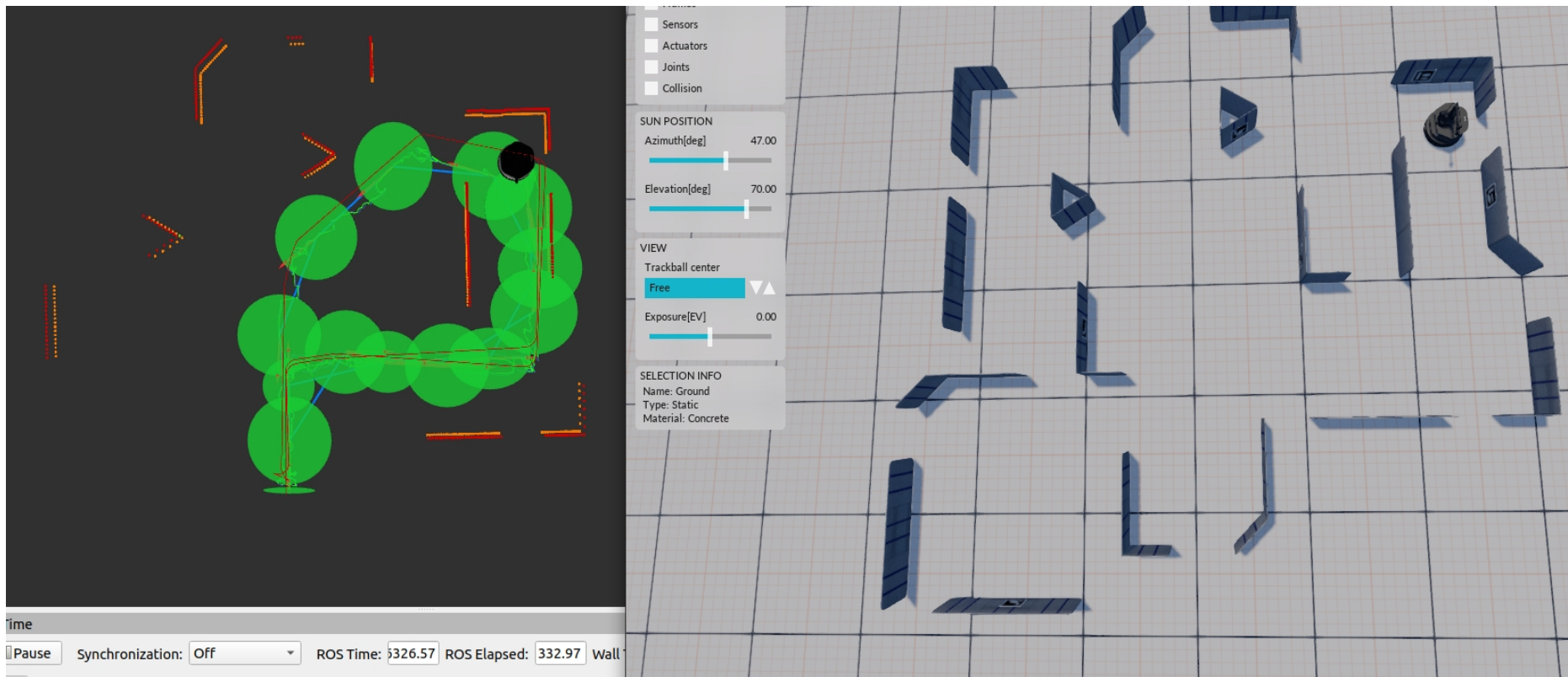
1. Voxelization (0.02 m)
2. Probability Distributions
3. Optimization
 - * Step Size: 0.1
 - * Transformation Epsilon: 0.01

EKF Update Step
(Fuse NDT observation with prediction)

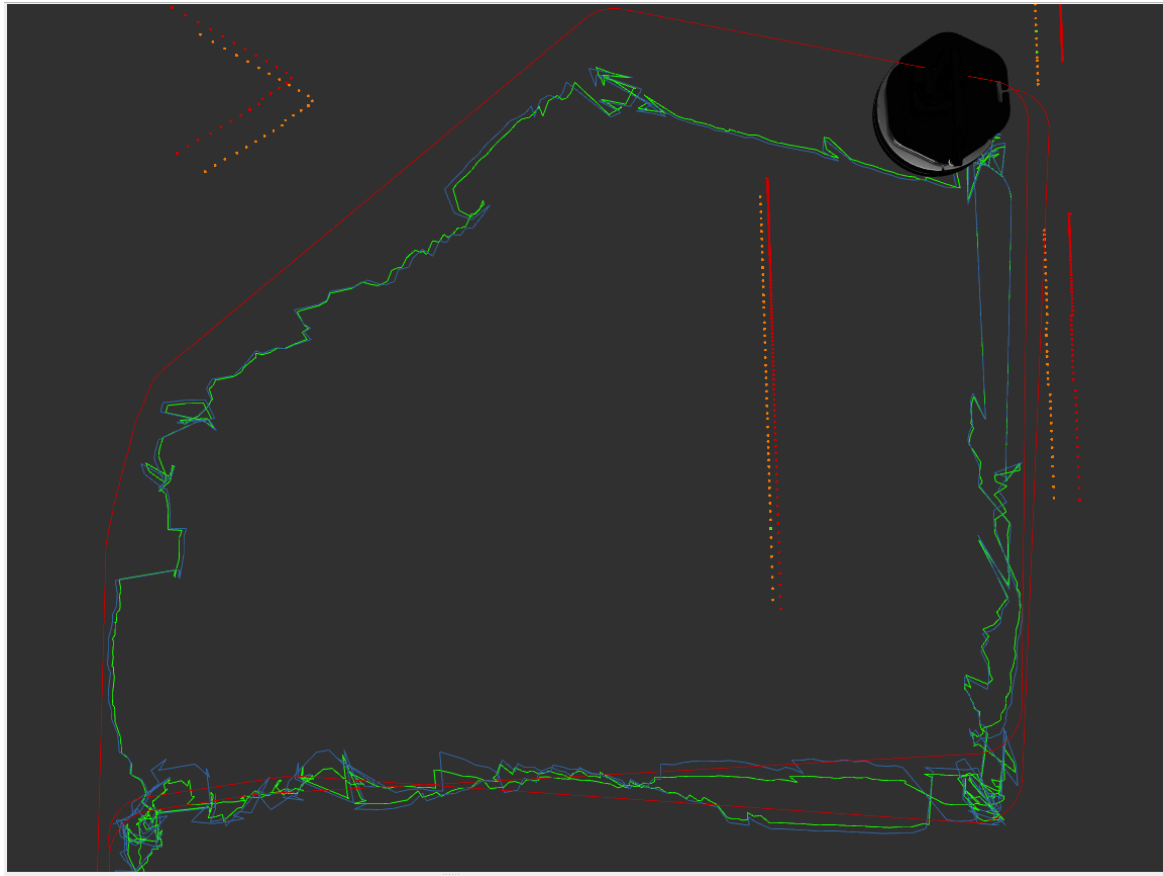
Loop Closure Pose-based EKF with NDT

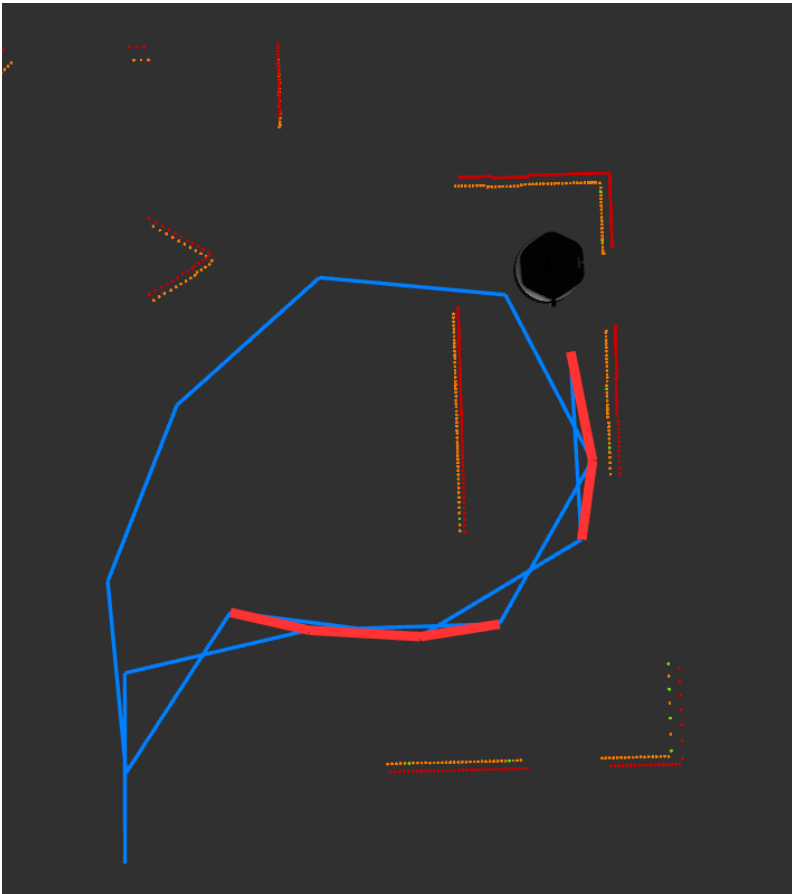


Simulation Environment and Robot Trajectory



Odometry path vs NDT path vs EKF path





```
[INFO] [1748125993.141042]: Started controllers: joint_state_controller, joint_velocity_controller
[ INFO] [1748125993.150508106]: Initialized GT pose: x=3.00, y=-0.78, yaw=90.00
[ WARN] [1748125993.347640199]: First scan received, setting as target.
[WARN] [1748125993.724790]: Saved keyframe #0 at (3.00, -0.78)
[ INFO] [1748125993.727654174]: Keyframe received: pose = (3.00, -0.78, 1.57)
[WARN] [1748126024.025419]: Saved keyframe #1 at (3.01, 0.23)
[ INFO] [1748126024.028096720]: Keyframe received: pose = (3.01, 0.23, 1.55)
[WARN] [1748126056.805255]: Saved keyframe #2 at (2.03, 0.46)
[ INFO] [1748126056.807157963]: Keyframe received: pose = (2.03, 0.46, -3.11)
[WARN] [1748126072.368848]: Saved keyframe #3 at (1.02, 0.50)
[ INFO] [1748126072.371708615]: Keyframe received: pose = (1.02, 0.50, 3.12)
[WARN] [1748126092.358665]: Saved keyframe #4 at (0.52, 1.37)
[ INFO] [1748126092.361223138]: Keyframe received: pose = (0.52, 1.37, 1.59)
[WARN] [1748126112.768734]: Saved keyframe #5 at (0.99, 2.26)
[ INFO] [1748126112.771176963]: Keyframe received: pose = (0.99, 2.26, 0.27)
[WARN] [1748126128.393339]: Saved keyframe #6 at (1.98, 2.34)
[ INFO] [1748126128.396660056]: Keyframe received: pose = (1.98, 2.34, -0.71)
[WARN] [1748126140.617357]: Saved keyframe #7 at (2.74, 1.66)
[ INFO] [1748126140.618708810]: Keyframe received: pose = (2.74, 1.66, -0.67)
[WARN] [1748126162.187244]: Saved keyframe #8 at (3.11, 0.71)
[ INFO] [1748126162.189437102]: Keyframe received: pose = (3.11, 0.71, -1.53)
[WARN] [1748126173.794765]: Saved keyframe #9 at (3.00, -0.30)
[ INFO] [1748126173.796873780]: Keyframe received: pose = (3.00, -0.30, -1.55)
[WARN] [1748126220.170987]: Saved keyframe #10 at (2.45, 0.55)
[ INFO] [1748126220.175404254]: Keyframe received: pose = (2.45, 0.55, 3.13)
[ WARN] [1748126220.229438916]: Loop closure: #10 ??? #2 | fitness=0.159 | overlap=0.80 | maha=1.15
[WARN] [1748126220.229509]: Loop correction applied: state_history[10] and state_history[2] updated.
[WARN] [1748126231.619324]: Saved keyframe #11 at (1.44, 0.43)
[ INFO] [1748126231.622801057]: Keyframe received: pose = (1.44, 0.43, -3.10)
[ WARN] [1748126231.653601817]: Loop closure: #11 ??? #2 | fitness=0.267 | overlap=0.59 | maha=2.00
[WARN] [1748126231.653798]: Loop correction applied: state_history[11] and state_history[2] updated.
[ WARN] [1748126231.680620556]: Loop closure: #11 ??? #3 | fitness=0.330 | overlap=0.49 | maha=1.09
[WARN] [1748126231.680732]: Loop correction applied: state_history[11] and state_history[3] updated.
[WARN] [1748126254.569377]: Saved keyframe #12 at (0.58, 0.95)
[ INFO] [1748126254.572755053]: Keyframe received: pose = (0.58, 0.95, 1.56)
[ WARN] [1748126254.635667764]: Loop closure: #12 ??? #4 | fitness=0.496 | overlap=0.78 | maha=0.97
[WARN] [1748126254.636030]: Loop correction applied: state_history[12] and state_history[4] updated.
[WARN] [1748126265.148202]: Saved keyframe #13 at (0.64, 1.95)
[ INFO] [1748126265.152038823]: Keyframe received: pose = (0.64, 1.95, 1.53)
[ WARN] [1748126265.180535227]: Loop closure: #13 ??? #4 | fitness=0.779 | overlap=0.52 | maha=1.91
[WARN] [1748126265.180695]: Loop correction applied: state_history[13] and state_history[4] updated.
```

Hardware Testing of PEKF-SLAM with NDT (TurtleBot + Lidar)

Test Setup:

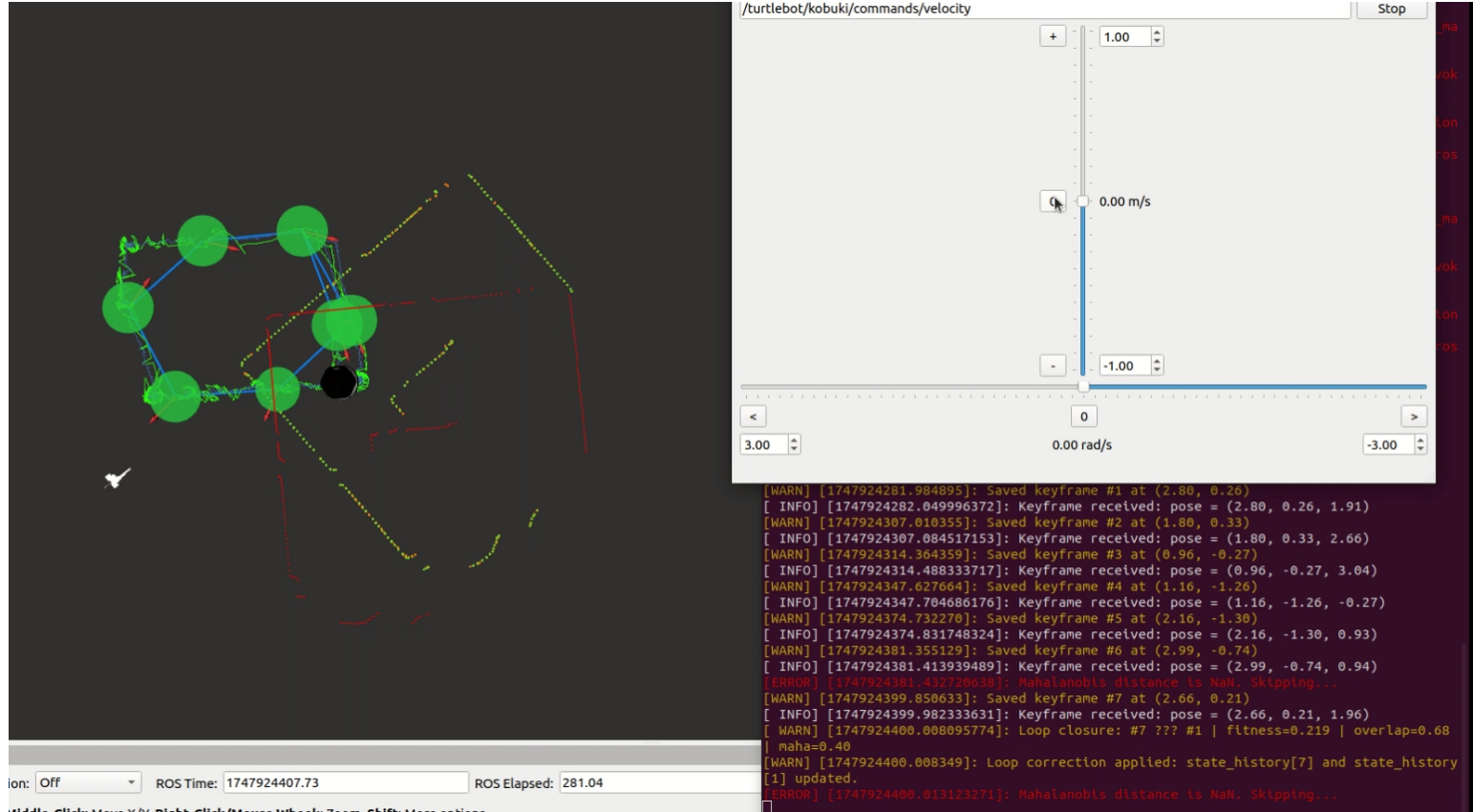
- Real TurtleBot tested in a square loop around a **central obstacle** inside a **bounded hallway**.
- Purpose: Validate **loop closure detection** in real-world conditions.

Key Observations:

1. **Loop closure worked effectively** even in a highly repetitive environment.
2. Robust due to **3-step validation**:
 - Fitness Score > 0.2
 - Overlap > 0.6
 - Mahalanobis Distance check
3. Prevented **false positives** despite visual similarities.

Issues Faced on Real Robot: Unexpected **TF frame behavior** not seen in simulation.
During robot rotation, scans rotated along with the robot.

Hardware Implementations



The image displays a ROS2 interface for a TurtleBot3 robot. On the left, a SLAM map is visible, showing the robot's trajectory (dashed yellow line) and its current position (black circle). The map includes several green circular landmarks and red lines representing the robot's path. On the right, a velocity control panel is shown, titled "/turtlebot/kobuki/commands/velocity". It features a vertical slider for linear velocity (m/s) and a horizontal slider for angular velocity (rad/s). The linear velocity is currently set to 0.00 m/s, and the angular velocity is set to 0.00 rad/s. Below the sliders, a terminal window displays ROS2 log messages, including keyframe saves, pose updates, and loop closure information.

Keyframe received: pose = (2.80, 0.26, 1.91)
Keyframe received: pose = (1.80, 0.33, 2.66)
Keyframe received: pose = (0.96, -0.27, 3.04)
Keyframe received: pose = (1.16, -1.26, -0.27)
Keyframe received: pose = (2.16, -1.30, 0.93)
Keyframe received: pose = (2.99, -0.74, 0.94)
Keyframe received: pose = (2.66, 0.21, 1.96)
Loop closure: #7 ??? #1 | fitness=0.219 | overlap=0.68
Loop correction applied: state_history[7] and state_history[1] updated.

ROS Time: 1747924407.73 ROS Elapsed: 281.04

Displays

✓ Fixed Frame

Grid

LaserScan

PointCloud2

PointCloud2

MarkerArray

✓ Status: Ok

Marker Topic

Queue Size

Namespaces

✓ Path

✓ Status: Ok

Topic

Unreliable

Queue Size

Line Style

Color

Alpha

Buffer Length

Offset

Pose Style

✓ Path

✓ Status: Ok

Topic

Unreliable

Queue Size

Line Style

Color

Alpha

Buffer Length

Offset

Pose Style

⚠ Marker

⚠ Status: Warn

Marker Topic

Queue Size

Namespaces

⚠ MarkerArray

OK

☐

☒

☒

☒

☐

/ekf_keyframe_ellipses

100

☒

/ekf_path

☐

10

Lines

52; 101; 164

1

1

0; 0; 0

None

☒

/odom_ndt_path

☐

10

Lines

25; 255; 0

1

1

0; 0; 0

None

☒

/odometry_edges

100

☒

Global Status: Ok

Add

Duplicate

Remove

Rename

The figure displays a 2D visualization of a robot's trajectory and keyframes. The trajectory is shown as a series of blue lines forming a closed loop, with red segments indicating keyframes. Green lines represent the laser scan data (PointCloud2) showing the environment's boundaries. The visualization is titled "Displays" and includes a "Global Status: Ok" indicator.

30

Problem Statement

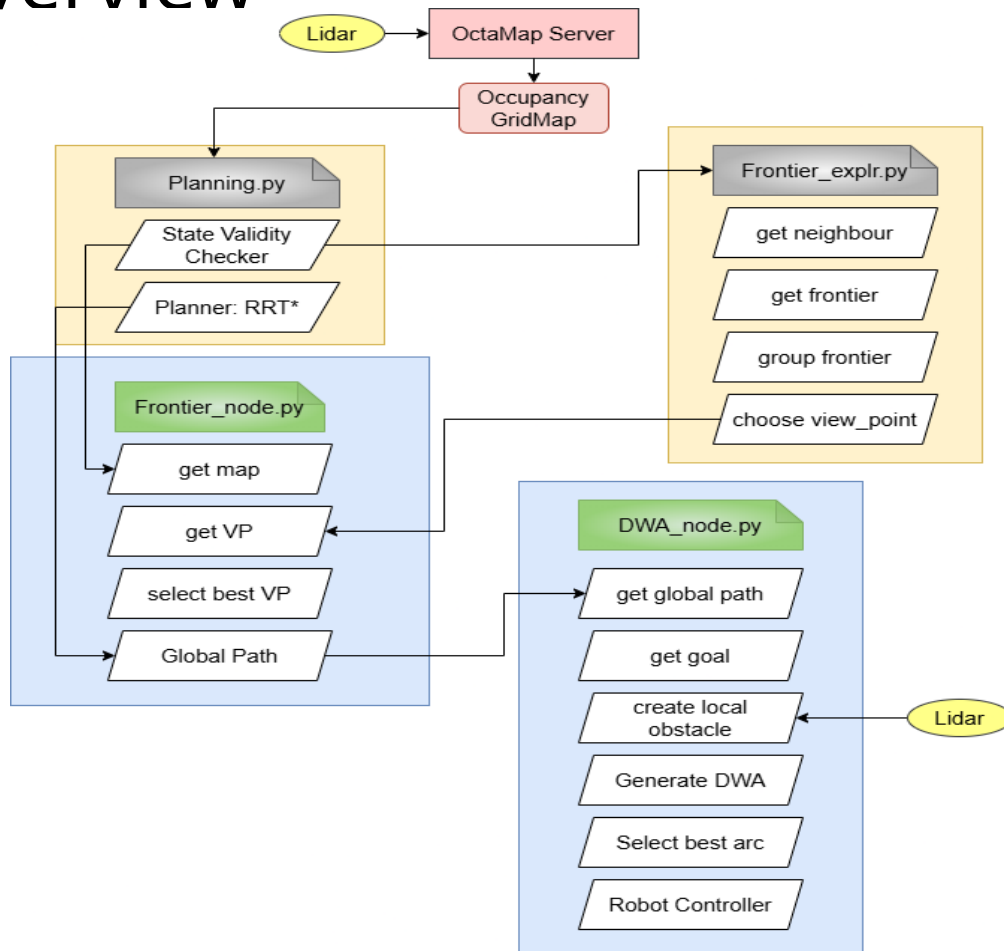
Challenge:

- Enable a TurtleBot to autonomously explore an unknown environment and build a map using a LiDAR sensor and occupancy grid.

Key Features:

- Identifying **frontier regions** (unknown-occupied boundaries)
- Planning **collision-free global paths**
- Tracking paths robustly in real-time with **obstacle avoidance**
- Ensuring **dynamic re-planning** when stuck or map is unchanged

System Overview



Frontier Detection

Goal: Detect unexplored frontiers and pick the best one to go toward.

Steps:

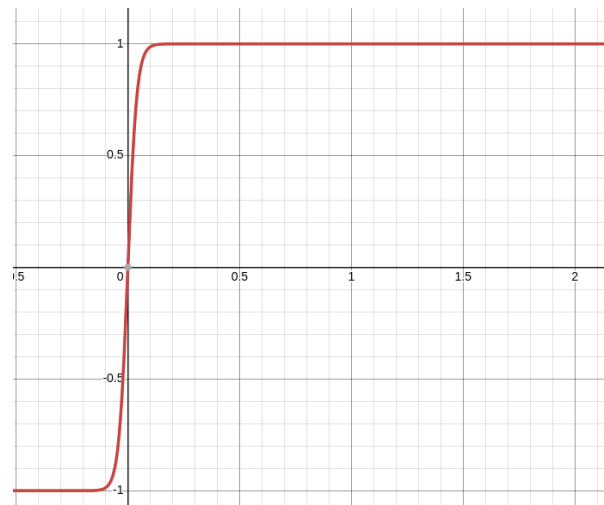
- Read the occupancy grid:
 - Identify frontiers
- Use Breadth-First Search (BFS) to group frontiers
- Score each group with weights:
 - Based on:
 - Distance from robot
 - Size of group
- Choose the group with the highest score.
- Compute a viewpoint cell and find nearest non-colliding

DWA (Dynamic Window Approach) – Local Control

Goal: From current pose, find a velocity command (v , w) that safely moves the robot toward the goal.

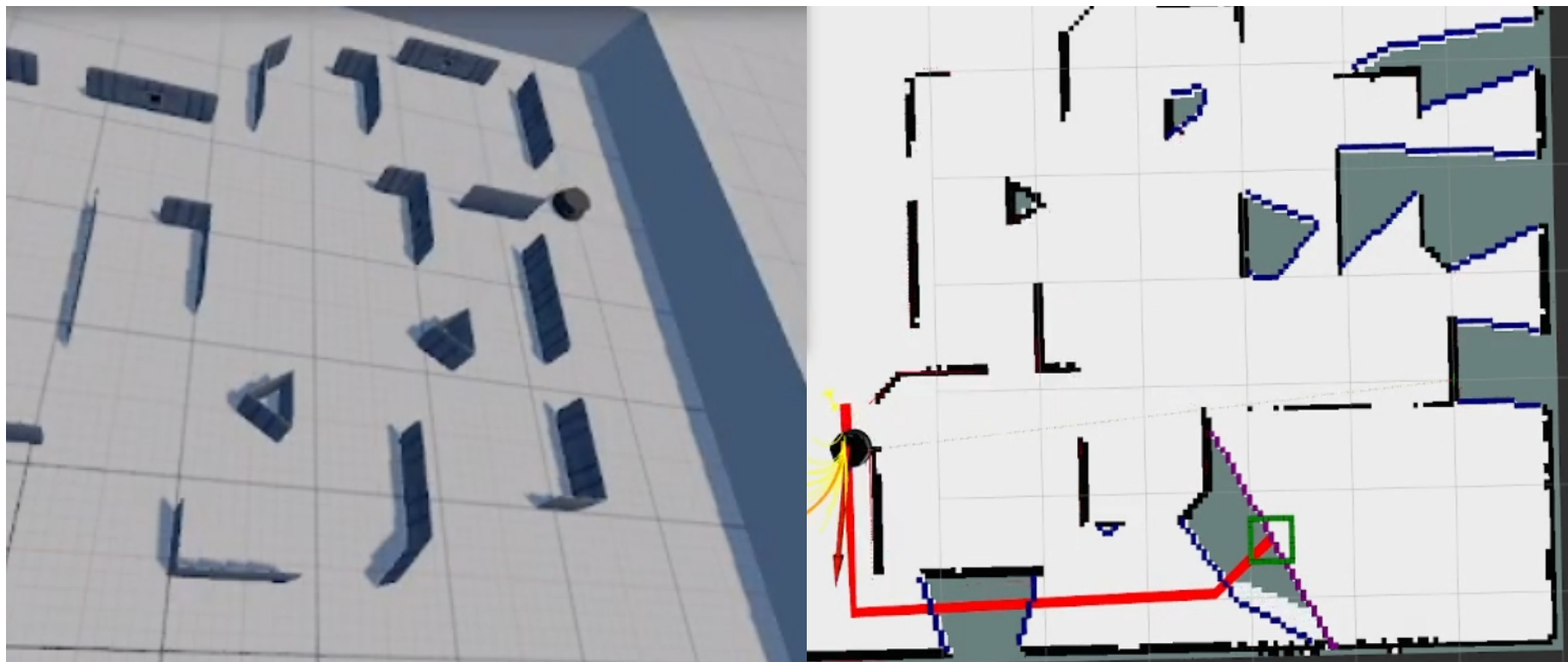
Steps:

- Generate possible velocities:
 - Based on current speed, acceleration limits, and robot constraints
- For each velocity:
 - Simulate robot motion over a short time window
 - Predict trajectory (arc-like path)
 - Score the trajectory using weights:
 - Heading, Velocity, Distance, Clearance
- Send linear and angular velocity of best score to controller
- Visual markers are published to RViz for debugging:
 - All arcs, best arc, and obstacle locations

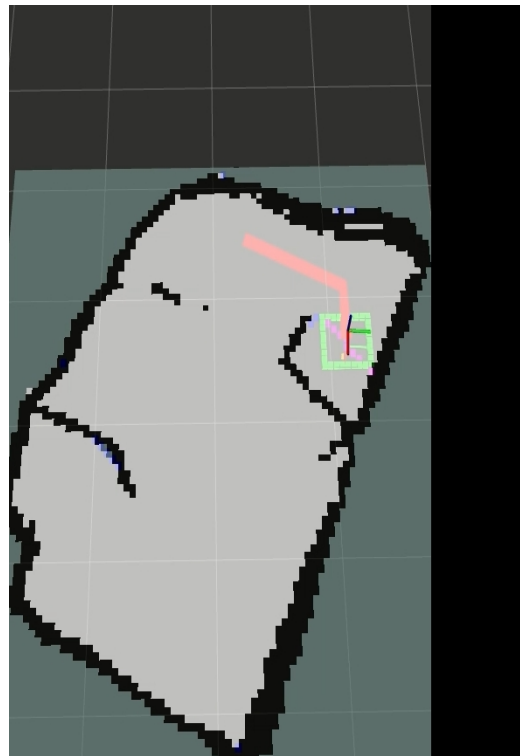
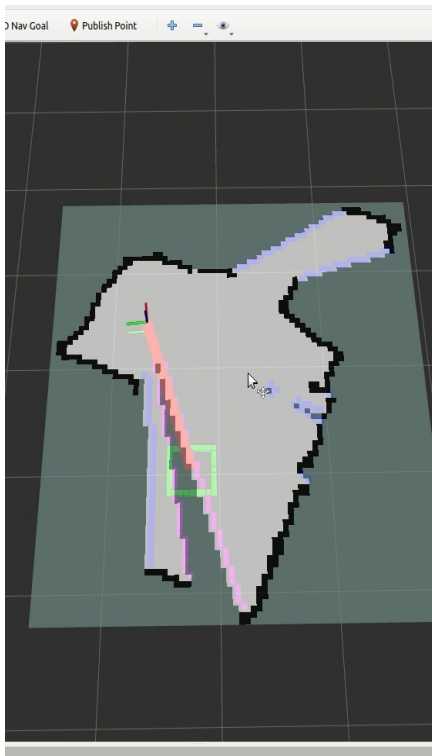


$$y = \frac{2}{1 + e^{-50x}} - 1$$

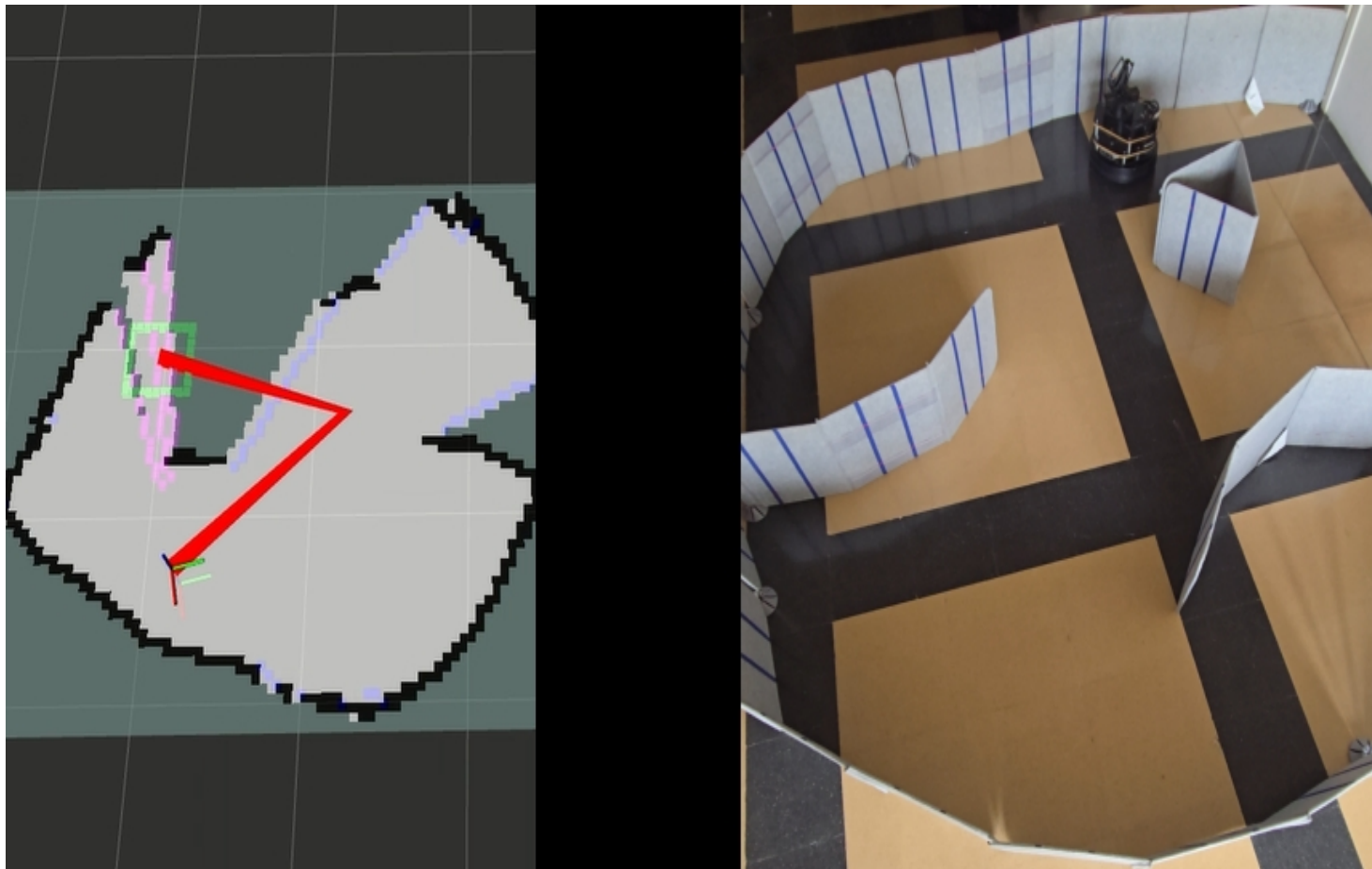
Simulation Results



Demonstration & Results (Map 1)



Demonstration & Results (Map 2)



Challenges Faced During Hardware Testing

Issues	Solution Implemented
Velocity Deadband	● Increased minimum linear and angular velocity values ● Tuned DWA velocity weights and clipping thresholds
Sim_time mismatch	● In simulation, sim_time = 4s worked. But, in real robot, prediction beyond 1s was unstable. ● High computation power to compute velocities causing delays in the robot movements based on the velocity signals received.
False Lidar Obstacle Detection	● Lidar detected its own wires as obstacles. ● Increased the obstacle clearance radius in DWA ● Added minimum distance threshold to ignore such noise.
Global Planner Efficiency	KD-Trees for efficient nearest-neighbor search